

Python E-Course

Module 2 — Image I/O & Display

Detailed Notes

Module Overview

Level: Foundations

Duration: ~1 week

Prerequisites: Module 1

What You'll Learn

- Read images with `cv2.imread` (and handle `None` returns).
- Write images with `cv2.imwrite` and tune format-specific parameters.
- Display images correctly with OpenCV's HighGUI vs `matplotlib`.
- Pick the right format (JPEG vs PNG vs BMP vs TIFF) for the job.
- Work with Pillow (PIL) and interop with NumPy / OpenCV.

Lessons

#	Lesson	Duration
1	Reading Images	30 min
2	Writing Images	25 min
3	Displaying with OpenCV	30 min
4	Displaying with Matplotlib	25 min
5	Image Formats Deep Dive	30 min
6	Working with Pillow	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 3: Pixel Manipulation

Lesson 1: Reading Images

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Use `cv2.imread` and handle `None` returns safely.
- Choose the right `IMREAD_*` flag for grayscale, color, alpha, or 16-bit images.
- Read images from filesystem paths and from byte buffers.

Prerequisites

- Module 1

1. Why This Matters

Half of beginner image-processing bugs trace back to `cv2.imread` returning `None` silently (typo in path, unsupported format) or to picking the wrong flag (you wanted alpha, got 3 channels). Five minutes here saves hours later.

2. Concept

2.1 Signature

```
img = cv2.imread(filename, flags=cv2.IMREAD_COLOR)
```

Returns a NumPy array on success, `None` on failure (bad path, unreadable format). It does NOT raise.

2.2 The flags

Flag	Behaviour	Output shape
<code>cv2.IMREAD_COLOR</code> (default)	Always 3-channel BGR; drops alpha	(H, W, 3) uint8
<code>cv2.IMREAD_GRAYSCALE</code>	Convert to grayscale on load	(H, W) uint8
<code>cv2.IMREAD_UNCHANGED</code>	As-is (preserves alpha, 16-bit)	varies
<code>cv2.IMREAD_ANYDEPTH</code>	Keep bit depth if >8	(H, W) uint8 or uint16
<code>cv2.IMREAD_REduced_COLOR_2</code>	Color, decoded at half resolution	smaller
<code>cv2.IMREAD_REduced_GRAYSCALE_4</code>	Grayscale at quarter resolution	smaller

When in doubt: `IMREAD_UNCHANGED` preserves what's in the file (alpha, 16-bit depth).

2.3 Always check for None

```
img = cv2.imread("photo.jpg")
if img is None:
```

```
        raise FileNotFoundError("Couldn't read 'photo.jpg' (bad path or unsupported
format)")
```

Skip this and the next line will crash with a much less helpful `AttributeError: 'NoneType' object has no attribute 'shape'`.

2.4 What imread decodes

OpenCV ships with codecs for JPEG, PNG, BMP, TIFF, WebP, PPM/PGM, and a few more. It does not read GIFs (use Pillow), proprietary RAW formats (use rawpy), or HEIC/HEIF on most platforms (use Pillow + plugin).

2.5 Reading from bytes (e.g. HTTP response)

```
import numpy as np
import cv2

with open("photo.jpg", "rb") as f:
    raw = f.read()
    arr = np.frombuffer(raw, dtype=np.uint8)
    img = cv2.imdecode(arr, cv2.IMREAD_COLOR)
```

Useful when you fetched the image from requests or a database BLOB.

2.6 Unicode paths on Windows

`cv2.imread` doesn't support non-ASCII paths on Windows. Workaround:

```
import numpy as np
import cv2
img = cv2.imdecode(np.fromfile("привет.jpg", dtype=np.uint8), cv2.IMREAD_COLOR)
```

`np.fromfile` handles the Unicode filename; `cv2.imdecode` does the decoding.

3. Code Examples

Example 1: Basic load with None check

```
import cv2

path = "datasets/starter/astronaut.png"
img = cv2.imread(path)
if img is None:
    raise FileNotFoundError(path)

print("shape:", img.shape, "dtype:", img.dtype)
```

Expected Output:

```
shape: (512, 512, 3) dtype: uint8
```

Example 2: Compare flags

```
import cv2

path = "datasets/starter/coffee.png"    # color png
```

```

for flag, name in [
    (cv2.IMREAD_COLOR, "COLOR"),
    (cv2.IMREAD_GRAYSCALE, "GRAYSCALE"),
    (cv2.IMREAD_UNCHANGED, "UNCHANGED"),
]:
    img = cv2.imread(path, flag)
    print(f"{name:<10} shape={img.shape} dtype={img.dtype}")

```

Expected Output (shapes for a 3-channel png):

```

COLOR      shape=(400, 600, 3) dtype=uint8
GRAYSCALE  shape=(400, 600)   dtype=uint8
UNCHANGED  shape=(400, 600, 3) dtype=uint8

```

For a PNG with alpha, IMREAD_COLOR would still give (H, W, 3) (alpha discarded), while IMREAD_UNCHANGED gives (H, W, 4).

Example 3: Decode from bytes

```

import cv2
import numpy as np

with open("datasets/starter/coins.png", "rb") as f:
    raw = f.read()

arr = np.frombuffer(raw, dtype=np.uint8)
img = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)
print("shape:", img.shape, "dtype:", img.dtype)

```

Expected Output:

```

shape: (303, 384) dtype: uint8

```

Example 4: Robust loader function

```

import cv2
import numpy as np
from pathlib import Path

def load(path, mode="color"):
    flag = {"color": cv2.IMREAD_COLOR,
            "gray": cv2.IMREAD_GRAYSCALE,
            "asis": cv2.IMREAD_UNCHANGED}[mode]
    # Unicode-safe on Windows
    img = cv2.imdecode(np.fromfile(str(path), dtype=np.uint8), flag)
    if img is None:
        raise FileNotFoundError(f"can't load: {path}")
    return img

print(load("datasets/starter/camera.png", "gray").shape)

```

Expected Output: (512, 512)

4. Parameter Sensitivity

Flag	When to use
IMREAD_COLOR	Standard photos, you don't care about alpha
IMREAD_GRAYSCALE	Edge detection, OCR, simple thresholding
IMREAD_UNCHANGED	Need alpha, or processing 16-bit medical / satellite data
IMREAD_ANYDEPTH	16-bit grayscale specifically

5. Common Mistakes

Mistake	What Happens	Fix
No if img is None check	Crash on img.shape with confusing error	Always check
Path typo	Silent None return	None check + raise
Reading PNG-with-alpha as COLOR	Alpha silently dropped	Use IMREAD_UNCHANGED
Reading 16-bit as default	Quantized to uint8	Use IMREAD_UNCHANGED
Non-ASCII Windows path	None return	cv2.imdecode + np.fromfile

6. Practice Tasks

- Write a `safe_imread(path)` that raises `FileNotFoundError` if OpenCV returns `None`.
- Read `astronaut.png` three ways (color, grayscale, unchanged). Print each shape and dtype.
- Load an image from raw bytes by first reading the file with `open(..., 'rb')`, then `cv2.imdecode`.

7. Key Takeaways

- `cv2.imread` returns `None` silently on failure — always check.
- Pick the flag deliberately: `COLOR`, `GRAYSCALE`, or `UNCHANGED`.
- `cv2.imdecode(np.fromfile(...))` handles Unicode paths and byte buffers.

8. What's Next

Writing images — with format-specific quality/compression settings.

Lesson 2: Writing Images

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Use `cv2.imwrite` and read its return value.
- Tune JPEG quality and PNG compression.
- Encode an image to a byte buffer with `cv2.imencode`.

Prerequisites

- Module 2 Lesson 1

1. Why This Matters

You'll save images constantly — intermediate stages, debug outputs, final results. Wrong format = blocky JPEGs of pristine masks, or 10× larger files than needed.

2. Concept

2.1 Signature

```
ok = cv2.imwrite(filename, img, params=None)
```

Returns True / False. Format is inferred from the file extension (.jpg, .png, .bmp, .tif, .webp, ...). Image must be BGR or grayscale uint8 unless you use a format that supports more.

2.2 Format-specific params

Passed as a flat list [FLAG_ID, value, FLAG_ID, value, ...]:

Format	Param flag	Range	Effect
JPEG	<code>cv2.IMWRITE_JPEG_QUALITY</code>	0–100	Higher = better quality, bigger file
PNG	<code>cv2.IMWRITE_PNG_COMPRESSION</code>	0–9	Higher = smaller file, slower write
WebP	<code>cv2.IMWRITE_WEBP_QUALITY</code>	0–100	Same as JPEG
TIFF	<code>cv2.IMWRITE_TIFF_COMPRESSION</code>	enum	LZW, none, etc.
PNG	<code>cv2.IMWRITE_PNG_STRATEGY</code>	enum	Compression strategy

```
cv2.imwrite("photo.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])  
cv2.imwrite("photo.png", img, [cv2.IMWRITE_PNG_COMPRESSION, 9])
```

2.3 Common pitfalls

- Floating-point images: convert to uint8 first (clip + astype).
- 16-bit images: PNG and TIFF support 16-bit; JPEG doesn't (it'll truncate).
- Always check the return value if writing to a network share or USB.

2.4 Encode to bytes

For uploading or storing in a database:

```
ok, buf = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 85])
if ok:
    data = buf.tobytes()
```

buf is a NumPy array of uint8; .tobytes() gives raw bytes you can shove anywhere.

2.5 Atomic writes

Half-written files happen if the process dies mid-save. Write to a temp file then rename:

```
from pathlib import Path
import os
tmp = Path("out.jpg.tmp")
cv2.imwrite(str(tmp), img)
os.replace(tmp, "out.jpg")    # atomic on POSIX & Windows
```

3. Code Examples

Example 1: Save with different qualities

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

for q in [10, 50, 90]:
    fname = f"coffee_q{q}.jpg"
    ok = cv2.imwrite(fname, img, [cv2.IMWRITE_JPEG_QUALITY, q])
    print(f"q={q:2d}  ok={ok}  size={os.path.getsize(fname)/1024:6.1f} KB")
```

Expected Output:

```
q=10  ok=True  size=   8.4 KB
q=50  ok=True  size=  22.9 KB
q=90  ok=True  size=  60.7 KB
```

Visual: q=10 has visible blocky / blotchy artefacts; q=90 looks identical to original at normal viewing distance.

Example 2: PNG compression levels

```
import cv2
import os
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)

for c in [0, 3, 6, 9]:
    fname = f"astro_c{c}.png"
    cv2.imwrite(fname, img, [cv2.IMWRITE_PNG_COMPRESSION, c])
    print(f"compression={c}  size={os.path.getsize(fname)/1024:7.1f} KB")
```


Expected Output (sizes roughly):

```
compression=0  size= 790.1 KB
compression=3  size= 470.2 KB
compression=6  size= 410.8 KB
compression=9  size= 392.3 KB
```

Example 3: Float image → save correctly

```
import cv2
import numpy as np

# A float image in [0, 1]
img_f = np.linspace(0, 1, 256, dtype=np.float32)
img_f = np.tile(img_f, (100, 1))  # (100, 256)

# Wrong: save as-is – PNG interprets float weirdly or fails
cv2.imwrite("wrong.png", img_f)
print("wrong size:", __import__("os").path.getsize("wrong.png"))

# Right: scale and cast
img_u = (img_f * 255).clip(0, 255).astype(np.uint8)
cv2.imwrite("right.png", img_u)
print("right size:", __import__("os").path.getsize("right.png"))
```

Expected Output: "wrong.png" file will exist but typically all-black or all-white when reloaded.
"right.png" is a proper horizontal gradient.

Example 4: Encode to bytes (e.g., for upload)

```
import cv2
from skimage.data import chelsea

img = cv2.cvtColor(chelsea(), cv2.COLOR_RGB2BGR)
ok, buf = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 80])
print("ok:", ok, " bytes:", len(buf.tobytes()))

# Round-trip:
import numpy as np
back = cv2.imdecode(np.frombuffer(buf.tobytes(), np.uint8), cv2.IMREAD_COLOR)
print("decoded shape:", back.shape)
```

Expected Output:

```
ok: True  bytes: 40132
decoded shape: (300, 451, 3)
```

4. Parameter Sensitivity

Param	Low	High
JPEG quality (0..100)	Smaller file, blocky	Big file, near-lossless
PNG compression (0..9)	Big file, fastest	Small file, slow write

5. Common Mistakes

Mistake	What Happens	Fix
Saving float (0..1) to PNG	All black	Multiply by 255, clip, cast to uint8
Ignoring return value	Silent failures on bad paths	Check ok
JPEG for masks / line art	Blurry / blocky	Use PNG (lossless) for non-photographic content
Wrong extension	Format mismatch	Match extension to actual format

6. Practice Tasks

- Save coffee() at JPEG quality 5 and quality 95. Display both and compare.
- Save the same image as PNG at compression 0 and 9. Compare sizes.
- Encode an image to JPEG bytes with cv2.imencode. Round-trip via cv2.imdecode.

7. Key Takeaways

- cv2.imwrite returns True/False — check it.
- Format params via [FLAG_ID, value] pairs.
- Float → uint8 conversion before saving.
- cv2.imencode for buffers (network, DB).

8. What's Next

Displaying images with OpenCV's HighGUI (imshow / waitKey).

Lesson 3: Displaying with OpenCV

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Display an image with `cv2.imshow` and the required event loop.
- Use `cv2.waitKey` correctly.
- Handle window resizing and multiple windows.
- Recognize when HighGUI isn't available (headless, Jupyter).

Prerequisites

- Module 2 Lesson 2

1. Why This Matters

`cv2.imshow` is the fastest way to peek at an image during development — but it requires an event loop, doesn't work in Jupyter or headless servers, and silently fails in surprising ways. Knowing the rules saves frustration.

2. Concept

2.1 The minimal pattern

```
import cv2
img = cv2.imread("photo.jpg")
cv2.imshow("preview", img)
cv2.waitKey(0)                # pause until any key
cv2.destroyAllWindows()
```

Three rules:

- `imshow` creates / updates a named window. Same name = reused window.
- `waitKey(ms)` is required for the window to actually render. Without it, nothing shows.
- Always `destroyAllWindows()` to clean up.

2.2 `cv2.waitKey(ms)`

- 0 = wait forever until any key is pressed.
- positive int = wait that many milliseconds; returns -1 if no key was pressed.
- Returns the key code (an int) if a key was pressed.

```
key = cv2.waitKey(0) & 0xFF      # mask to get the actual key
if key == ord('q'):
    print("quit pressed")
```

The `& 0xFF` masks higher bits set by some keyboards / OSes.

2.3 Video / animation loop

```
cap = cv2.VideoCapture(0)
while True:
    ok, frame = cap.read()
    if not ok:
        break
    cv2.imshow("camera", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

`waitKey(1)` waits ~1ms — long enough to update the window, short enough to keep the framerate high.

2.4 Multiple windows

```
cv2.imshow("original", img)
cv2.imshow("blurred", cv2.blur(img, (5, 5)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2.5 Resizable windows

```
cv2.namedWindow("win", cv2.WINDOW_NORMAL) # allows resizing
cv2.resizeWindow("win", 800, 600)
cv2.imshow("win", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Default windows fit the image; `WINDOW_NORMAL` lets the user drag-resize.

2.6 When imshow doesn't work

- Headless server / WSL without X: HighGUI crashes or hangs. Use `matplotlib` (next lesson) or `opencv-python-headless`.
- Jupyter Notebook: `imshow` opens a window outside the notebook — usually fails. Use `plt.imshow` inside notebooks.
- macOS: Requires the cocoa backend; usually fine with `opencv-python` from pip.

3. Code Examples

Example 1: Show one image

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
cv2.imshow("astronaut", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output (visual): A window titled "astronaut" pops up showing the portrait. Window closes when you press any key.

Example 2: Compare two side-by-side via stacking

```
import cv2
import numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
blur = cv2.blur(img, (15, 15))

combined = np.hstack([img, blur])
cv2.imshow("orig | blur", combined)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output: Single window showing original on the left, heavily blurred version on the right.

Example 3: Press-q-to-quit loop

```
import cv2
from skimage.data import camera

img = camera()
while True:
    cv2.imshow("press q", img)
    key = cv2.waitKey(50) & 0xFF
    if key == ord('q'):
        break
cv2.destroyAllWindows()
print("done")
```

Expected Output: Window stays open; closes only when you press 'q'.

Example 4: Resizable window for big images

```
import cv2
img = cv2.imread("datasets/starter/astronaut.png")

cv2.namedWindow("astronaut", cv2.WINDOW_NORMAL)
cv2.resizeWindow("astronaut", 1000, 1000)
cv2.imshow("astronaut", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output: A 1000×1000 window — the image is upscaled to fit; you can drag-resize the window further.

4. Parameter Sensitivity

waitKey(ms)	Use case
0	One-shot preview
1	Live video loop
30+	Slow-motion playback
Skipped	Window never renders — common bug

5. Common Mistakes

Mistake	What Happens	Fix
No waitKey	Window appears for an instant, then disappears	Always call waitKey(0) for stills
cv2.imshow in Jupyter	Window can't display, kernel hangs	Use plt.imshow (next lesson)
cv2.imshow on headless server	Crashes or freezes	Use opencv-python-headless + matplotlib / save to file
Comparing cv2.waitKey() directly to ord('q')	Unicode codes can have extra bits	Use & 0xFF mask

6. Practice Tasks

- Show coffee() with cv2.imshow. Wait for 'q' before closing.
- Build a 2-up side-by-side display with np.hstack and show with cv2.imshow.
- Use cv2.namedWindow(..., WINDOW_NORMAL) to make a resizable window.

7. Key Takeaways

- imshow + waitKey + destroyAllWindows is the trio.
- waitKey(1) for video loops; waitKey(0) for stills.
- cv2.imshow doesn't work in notebooks / headless — use matplotlib then.
- WINDOW_NORMAL for resizable windows.

8. What's Next

Displaying with matplotlib — the workhorse for notebooks and headless setups.

Lesson 4: Displaying with Matplotlib

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Display images correctly with `plt.imshow`.
- Convert BGR→RGB before display.
- Use appropriate colormap and value range for grayscale.
- Build multi-panel comparisons with subplots.

Prerequisites

- Module 2 Lesson 3

1. Why This Matters

In Jupyter notebooks and on headless servers, matplotlib is the display tool. It expects RGB and either float [0,1] or uint8 [0,255] — mix them wrong and you'll see garbage.

2. Concept

2.1 The basics

```
import matplotlib.pyplot as plt
plt.imshow(img)
plt.axis("off")
plt.show()
```

2.2 BGR → RGB

If `img` came from `cv2.imread`, it's BGR. Convert before display:

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

Without this, reds appear blue and vice versa.

2.3 Grayscale

For (H, W) arrays, matplotlib applies the default viridis colormap (greenish-purple). For traditional grayscale appearance:

```
plt.imshow(gray, cmap="gray", vmin=0, vmax=255)
```

`vmin/vmax` lock the display range so two images can be compared fairly.

2.4 dtype rules

Input dtype	Range matplotlib expects
uint8	0..255
float32/64	0.0..1.0

int	clipped to 0..1 or 0..255 — confusing
-----	---------------------------------------

If you have a float image not in [0, 1], divide first.

2.5 Multi-panel comparison

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(orig); axes[0].set_title("original"); axes[0].axis("off")
axes[1].imshow(blur); axes[1].set_title("blur"); axes[1].axis("off")
axes[2].imshow(edges, cmap="gray"); axes[2].set_title("edges"); axes[2].axis("off")
plt.tight_layout()
plt.show()
```

2.6 Saving the figure

```
plt.savefig("report.png", dpi=150, bbox_inches="tight")
```

For the underlying image itself, prefer `cv2.imwrite` — `plt.savefig` saves the whole figure (axes, titles).

3. Code Examples

Example 1: BGR vs RGB demo

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import astronaut

img_rgb = astronaut()
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

fig, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].imshow(img_bgr); ax[0].set_title("BGR shown as RGB (WRONG)");
ax[0].axis("off")
ax[1].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
ax[1].set_title("correctly converted"); ax[1].axis("off")
plt.show()
```

Expected Output (visual): Left panel has reds/oranges showing as blue/teal (the astronaut helmet looks wrong). Right panel has natural colors.

Example 2: Grayscale display with cmap

```
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
fig, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].imshow(img); ax[0].set_title("default cmap (viridis)"); ax[0].axis("off")
ax[1].imshow(img, cmap="gray", vmin=0, vmax=255); ax[1].set_title("gray cmap");
ax[1].axis("off")
plt.show()
```

Expected Output: Left shows the coin image tinted greenish-purple (viridis default). Right shows it in natural grayscale.

Example 3: Multi-panel grid

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(img, (15, 15), 0)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); ax[0].set_title("original")
ax[1].imshow(gray, cmap="gray"); ax[1].set_title("grayscale")
ax[2].imshow(cv2.cvtColor(blur, cv2.COLOR_BGR2RGB)); ax[2].set_title("blurred")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output (visual): Three-panel figure — color original, gray version, blurred version.

Example 4: Reusable helper

```
import cv2
import matplotlib.pyplot as plt

def show(img, title="", cmap=None):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(img)
    else:
        plt.imshow(img, cmap=cmap or "gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")
    plt.show()

img = cv2.imread("datasets/starter/coins.png", cv2.IMREAD_GRAYSCALE)
show(img, "coins")
```

4. Parameter Sensitivity

Param	Effect
cmap="gray"	Traditional grayscale (vs default viridis)
vmin/vmax	Lock display range; needed for fair comparisons
figsize=(w, h)	Panel size in inches at default DPI
dpi=N (in savefig)	Output resolution

5. Common Mistakes

Mistake	What Happens	Fix
Skipping BGR→RGB	Reds look blue	cv2.cvtColor(img, COLOR_BGR2RGB)
Default cmap on grayscale	Greenish tint	cmap="gray"
Float image not in [0,1]	All white / all clipped	Divide by max or use uint8
Two panels with auto-scaled	Brightness looks different	Set vmin=0, vmax=255 on

vmin/vmax	between panels	both
-----------	----------------	------

6. Practice Tasks

- Display `coins()` two ways: with default `cmap` and with `cmap="gray"`, `vmin=0`, `vmax=255`. Compare.
- Load `astronaut.png` via `cv2.imread`, display with matplotlib without BGR conversion. Note how it looks wrong.
- Build a 2×2 subplot showing: original color, grayscale, blurred, and edges.

7. Key Takeaways

- `plt.imshow(...)` for in-notebook / headless display.
- BGR → RGB conversion before display.
- `cmap="gray"` + `vmin=0`, `vmax=255` for grayscale.
- Multi-panel comparisons via `plt.subplots`.

8. What's Next

Image format deep dive — when to use which container.

Lesson 5: Image Formats Deep Dive

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick between JPEG, PNG, BMP, TIFF, WebP for the task.
- Recognize compression artefacts (blocky JPEGs).
- Understand which formats support alpha and 16-bit.

Prerequisites

- Module 2 Lesson 4

1. Why This Matters

JPEG-ing a binary mask is a great way to ruin it. Saving HDR data as a JPEG loses 8 bits. Knowing the format trade-offs saves rework and disk space.

2. Concept

2.1 Comparison table

Format	Compression	Lossy?	Alpha	Bit-depth	Best for
JPEG	DCT + quant	Yes	No	8	Photos
PNG	DEFLATE	No	Yes	8 or 16	Masks, line art, anything with sharp edges
BMP	None	No	Yes (rare)	8 / 24	Quick uncompressed dumps
TIFF	Many (LZW, ZIP, none)	Both	Yes	8 / 16 / 32	Scientific / publishing
WebP	Wavelet	Both (mode)	Yes	8	Modern web (smaller than JPEG/PNG)
GIF	LZW	Lossless*	No	8 (palette)	Animation; not for photos
HEIC/HEIF	HEVC	Yes	Yes	8/10	Apple devices; needs plugin

* GIF is technically lossless but limited to a 256-color palette — looks lossy on photos.

2.2 JPEG artefacts

JPEG splits the image into 8×8 blocks and quantises high-frequency content. At low quality, this causes:

- Blocking — visible 8×8 squares
- Ringing — wavy halos around sharp edges
- Color bleed — chroma subsampling

JPEG is terrible for screenshots with text and for binary masks — both rely on sharp edges.

2.3 PNG strengths

- Lossless — bit-exact round-trip
- Alpha channel
- 16-bit grayscale and 48-bit RGB
- Good compression on uniform regions (UI screenshots, masks)
- Bad on photos (still bigger than JPEG)

2.4 BMP

Almost no compression. Good for "quick dump that another tool will read instantly" or when you need raw pixel data with a header. Large file sizes.

2.5 TIFF

The Swiss Army knife. Optional compression, multiple pages, alpha, high bit-depth. Widely supported in scientific tools. OpenCV supports basic TIFF; for full features (multi-page, geo-TIFF), use specialized libraries.

2.6 WebP

Google's successor to JPEG+PNG. Both lossy and lossless modes. About 25-35% smaller than equivalent-quality JPEG. Now supported on all major browsers.

2.7 Rules of thumb

- Photo destined for web: JPEG q=80-90 or WebP.
- Mask / binary image: PNG.
- Intermediate pipeline output: PNG (lossless, no quality loss between stages).
- Scientific 16-bit data: PNG-16 or TIFF.
- Final deliverable for print: TIFF.

3. Code Examples

Example 1: Same image, three formats

```
import cv2, os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for ext in ["jpg", "png", "bmp"]:
    p = f"coffee.{ext}"
    cv2.imwrite(p, img)
    print(f"{ext:4s}: {os.path.getsize(p)/1024:7.1f} KB")
```

Expected Output:

```
jpg :    61.4 KB
png :   354.6 KB
bmp :   703.2 KB
```

Example 2: Demonstrate JPEG artefacts on a mask

```
import cv2, numpy as np

mask = np.zeros((200, 400), dtype=np.uint8)
cv2.putText(mask, "HELLO", (50, 130), cv2.FONT_HERSHEY_SIMPLEX, 3, 255, 8)

cv2.imwrite("mask.png", mask)
cv2.imwrite("mask.jpg", mask, [cv2.IMWRITE_JPEG_QUALITY, 30])

png_back = cv2.imread("mask.png", cv2.IMREAD_GRAYSCALE)
jpg_back = cv2.imread("mask.jpg", cv2.IMREAD_GRAYSCALE)

print("png unique values:", np.unique(png_back))    # [0, 255]
print("jpg unique values:", np.unique(jpg_back))    # many values - JPEG smeared
the edges
```

Expected Output:

```
png unique values: [0 255]
jpg unique values: [ 0  1  2 ... 253 254 255]
```

A binary mask saved as PNG round-trips exactly. As JPEG it becomes a smeary 0..255 gradient — useless for masking.

Example 3: PNG with alpha

```
import cv2, numpy as np

# Build RGBA image: red square with semi-transparent fade
img = np.zeros((200, 200, 4), dtype=np.uint8)
img[:, :, 2] = 255          # red channel (BGR order: index 2 = red)
img[:, :, 3] = np.tile(np.linspace(0, 255, 200, dtype=np.uint8), (200, 1))

cv2.imwrite("red_fade.png", img)

back = cv2.imread("red_fade.png", cv2.IMREAD_UNCHANGED)
print("loaded shape:", back.shape, "channels:", back.shape[2])
print("alpha min/max:", back[..., 3].min(), back[..., 3].max())
```

Expected Output:

```
loaded shape: (200, 200, 4) channels: 4
alpha min/max: 0 255
```

Example 4: 16-bit PNG round-trip

```
import cv2, numpy as np

img16 = (np.linspace(0, 65535, 256, dtype=np.uint16)
         .reshape(1, -1)
         .repeat(100, axis=0))
```

```
print("save dtype:", img16.dtype, "range:", img16.min(), img16.max())

cv2.imwrite("ramp16.png", img16)
back = cv2.imread("ramp16.png", cv2.IMREAD_UNCHANGED)
print("read dtype:", back.dtype, "range:", back.min(), back.max())
```

Expected Output:

```
save dtype: uint16 range: 0 65535
read dtype: uint16 range: 0 65535
```

4. Parameter Sensitivity

(Format is the parameter — see comparison table above.)

5. Common Mistakes

Mistake	What Happens	Fix
JPEG-ing a mask	Edges smear into gradients	Use PNG
Re-saving JPEG repeatedly	Quality degrades with each save	Store pipeline intermediates as PNG
Treating GIF as photo format	Banded color	Use JPEG / PNG / WebP
Saving 16-bit as JPEG	Truncated to 8-bit silently	Use PNG-16 or TIFF

6. Practice Tasks

- Save astronaut() as JPEG (q=85), PNG, BMP. Compare sizes.
- Save a binary mask as both JPEG and PNG. Reload and np.unique each.
- Save a 16-bit gradient as PNG and round-trip; verify range preserved.

7. Key Takeaways

- JPEG for photos; never for masks or screenshots.
- PNG is lossless and supports alpha + 16-bit.
- BMP for unconditional uncompressed dumps.
- TIFF for scientific / multi-page / high bit-depth.

8. What's Next

Pillow (PIL) — when and how to interop with the other major Python image library.

Lesson 6: Working with Pillow

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Open, convert, and save images with PIL.Image.
- Interop between Pillow and NumPy / OpenCV.
- Read formats OpenCV doesn't (GIF, HEIC with plugin).
- Use Pillow's text rendering (better fonts than cv2.putText).

Prerequisites

- Module 2 Lesson 5

1. Why This Matters

OpenCV is fast but limited (no GIF, awkward text). Pillow is slower but supports more formats, has nicer text rendering, and dominates the Python web/imaging ecosystem. Knowing both lets you pick the right tool.

2. Concept

2.1 Open / save

```
from PIL import Image
img = Image.open("photo.jpg")
print(img.size, img.mode)          # (W, H), 'RGB' / 'L' / 'RGBA' / ...
img.save("out.png")
```

Pillow returns its own Image object — not a NumPy array.

2.2 Mode

Mode	Meaning
L	8-bit grayscale
RGB	8-bit color
RGBA	8-bit color + alpha
1	1-bit binary
I;16	16-bit grayscale
CMYK	Print
P	Palette / GIF

Convert with `img.convert("L")`, etc.

2.3 Pillow ↔ NumPy

```
import numpy as np
from PIL import Image
```

```
# PIL -> NumPy
arr = np.array(Image.open("photo.jpg"))    # RGB by default
print(arr.shape, arr.dtype)                # (H, W, 3) uint8

# NumPy -> PIL
img = Image.fromarray(arr)                  # RGB
img.save("out.png")
```

2.4 Pillow ↔ OpenCV

Pillow is RGB; OpenCV is BGR. Convert at the boundary:

```
# PIL -> OpenCV
import cv2, numpy as np
from PIL import Image
pil = Image.open("p.jpg").convert("RGB")
cv2_img = cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)

# OpenCV -> PIL
cv2_img = cv2.imread("p.jpg")
pil = Image.fromarray(cv2.cvtColor(cv2_img, cv2.COLOR_BGR2RGB))
```

2.5 Text with Pillow (better fonts)

cv2.putText only does block Hershey fonts. Pillow uses TrueType:

```
from PIL import Image, ImageDraw, ImageFont

img = Image.new("RGB", (400, 200), (30, 30, 30))
draw = ImageDraw.Draw(img)
font = ImageFont.truetype("arial.ttf", 48)    # path varies by OS
draw.text((20, 60), "Hello, World!", font=font, fill=(255, 255, 255))
img.save("text.png")
```

2.6 Format coverage

Pillow reads:

- Everything OpenCV reads
- Plus GIF (including animated), ICO, PSD (limited), EPS, more legacy formats
- HEIC/HEIF if you install pillow-heif
- AVIF if you install pillow-avif-plugin

For animated GIF:

```
from PIL import Image
gif = Image.open("animated.gif")
for frame_index in range(gif.n_frames):
    gif.seek(frame_index)
    gif.copy().save(f"frame_{frame_index:03d}.png")
```


3. Code Examples

Example 1: PIL → NumPy round-trip

```
from PIL import Image
import numpy as np

pil = Image.open("datasets/starter/astronaut.png").convert("RGB")
arr = np.array(pil)
print("array:", arr.shape, arr.dtype)

# Modify (set top 20 rows to white)
arr[:20] = 255
pil2 = Image.fromarray(arr)
pil2.save("astro_top_white.png")
```

Expected Output (console):

```
array: (512, 512, 3) uint8
```

Visual: astronaut image with a white horizontal strip at the top.

Example 2: PIL → OpenCV pipeline

```
import cv2, numpy as np
from PIL import Image

pil = Image.open("datasets/starter/coffee.png").convert("RGB")
cv_img = cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)

blurred = cv2.GaussianBlur(cv_img, (15, 15), 0)
cv2.imwrite("coffee_blur.png", blurred)
```

Expected Output: coffee_blur.png — heavily-blurred coffee photo.

Example 3: TrueType text overlay

```
from PIL import Image, ImageDraw, ImageFont

img = Image.open("datasets/starter/astronaut.png").convert("RGB")
draw = ImageDraw.Draw(img)
try:
    font = ImageFont.truetype("arial.ttf", 36)
except OSError:
    font = ImageFont.load_default()
draw.rectangle((10, 460, 250, 510), fill=(0, 0, 0, 128))
draw.text((20, 470), "ASTRONAUT", font=font, fill=(255, 255, 255))
img.save("astro_caption.png")
```

Expected Output: Astronaut image with a small dark rectangle and "ASTRONAUT" label in the bottom-left.

Example 4: Save animated GIF

```
from PIL import Image
```

```
frames = [Image.new("RGB", (100, 100), (i * 25, 0, 255 - i * 25))
           for i in range(10)]
frames[0].save("anim.gif", save_all=True, append_images=frames[1:],
              duration=100, loop=0)
```

Expected Output: A 100×100 looping GIF where the color shifts from blue to red over 10 frames.

4. Parameter Sensitivity

Choice	Effect
convert("L")	grayscale
convert("RGBA")	adds alpha
convert("1")	binary, often lossy
Pillow vs OpenCV for resize	Pillow LANCZOS is slightly higher quality

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting BGR↔RGB at the boundary	Colors swap	Convert at the interface
Treating Pillow Image like a NumPy array	TypeError	Wrap with np.array(img) first
Pillow .size is (W, H)	Mixing with NumPy .shape (H, W)	Remember Pillow uses (W, H)
Trying to read HEIC without plugin	UnidentifiedImageError	pip install pillow-heif

6. Practice Tasks

- Open coffee.png with Pillow, convert to grayscale, save as PNG.
- Round-trip a NumPy array (e.g., random) through Image.fromarray and back to np.array. Confirm equality.
- Overlay TrueType text "DEMO" on astronaut.png using Pillow's ImageDraw.

7. Key Takeaways

- Pillow uses RGB (vs OpenCV's BGR) and (W, H) sizes.
- np.array(pil_img) and Image.fromarray(arr) round-trip cleanly.
- Pillow handles formats OpenCV doesn't (GIF, HEIC plugin, etc.).
- Pillow has better text rendering via ImageFont.truetype.

8. What's Next

End of Module 2. Next: Module 3 — Pixel Manipulation.

Module 2 — Exercises

18 exercises (3 per lesson).

2.1 Reading Images

- (E) Write a `safe_load(path)` that raises `FileNotFoundError` on `None`.
- (M) Read `coffee.png` 3 ways: `COLOR`, `GRAYSCALE`, `UNCHANGED`. Print each shape.
- (H) Decode an image from bytes — use `open('rb') + cv2.imdecode`.

2.2 Writing Images

- (E) Save `coffee.png` as JPEG at `q=10, 50, 95`. Compare file sizes.
- (M) Save the same image as PNG with compression 0 and 9. Compare sizes.
- (H) Encode an image to JPEG bytes with `cv2.imencode`. Round-trip via `cv2.imdecode`.

2.3 Display OpenCV

- (E) Show `coins.png` with `cv2.imshow`. Quit on `'q'`.
- (M) Side-by-side display: stack original and Gaussian-blurred with `np.hstack`.
- (H) Make a resizable window with `WINDOW_NORMAL` and size it to `1000×1000`.

2.4 Display Matplotlib

- (E) Display `coins.png` with `cmap="gray"`, `vmin=0`, `vmax=255`.
- (M) Load `astronaut.png` with `cv2.imread`; display with matplotlib with BGR conversion.
- (H) Build a `2×2` subplot grid: original, gray, blurred, edges.

2.5 Formats

- (E) Save `astronaut.png` as JPG, PNG, BMP. Compare sizes.
- (M) Save a binary mask as JPEG and PNG; reload and `np.unique` each.
- (H) Save a 16-bit gradient as PNG; reload and verify range.

2.6 Pillow

- (E) Open `coffee.png` with Pillow, convert to grayscale, save as PNG.
- (M) Round-trip a random NumPy array through Pillow.
- (H) Add TrueType caption text to `astronaut.png` using `ImageFont.truetype`.

Module 2 — Quiz

10 MCQs.

Q1

`cv2.imread("nope.jpg")` on a missing file: A. Raises `FileNotFoundError` B. Returns `None` C. Returns empty array D. Crashes

Answer: B (L1)

Q2

Which flag preserves alpha channel? A. `IMREAD_COLOR` B. `IMREAD_GRAYSCALE` C. `IMREAD_UNCHANGED` D. `IMREAD_ANYDEPTH`

Answer: C (L1)

Q3

`cv2.imwrite("x.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])` does: A. Saves at quality 90 B. Compresses 90% C. Resizes to 90% D. Errors

Answer: A (L2)

Q4

What's required for `cv2.imshow` to render? A. nothing B. `cv2.waitKey` C. `plt.show` D. `cv2.show`

Answer: B (L3)

Q5

For grayscale display in matplotlib, the right call is: A. `plt.imshow(g)` B. `plt.imshow(g, cmap="viridis")` C. `plt.imshow(g, cmap="gray", vmin=0, vmax=255)` D. `plt.gray(g)`

Answer: C (L4)

Q6

For a binary mask, the best format is: A. JPEG B. PNG C. GIF D. WebP-lossy

Answer: B (L5)

Q7

`PIL.Image.size` returns: A. (H, W) B. (W, H) C. (H, W, C) D. (W, H, C)

Answer: B (L6)

Q8

Going from Pillow (RGB) to OpenCV requires: A. nothing B. `cv2.cvtColor(..., COLOR_RGB2BGR)` C. transpose D. flip

Answer: B (L6)

Q9

cv2.waitKey(1) in a video loop: A. waits forever B. waits 1 sec C. waits 1 ms D. skips

Answer: C (L3)

Q10

Saving a float [0,1] image with cv2.imwrite without scaling produces: A. correct PNG B. all-black or all-white image C. error D. uint16 result

Answer: B (L2)

Module 2 — Assignment: Batch Image Format Converter

Estimated time: 1.5-2 hours

Combines: Module 2 + Module 1

Brief

Build a CLI tool that converts every image in a folder to a target format with specified quality / compression. Print a summary table.

Requirements

- `convert.py INPUT_DIR OUTPUT_DIR --format jpg|png|webp [--quality 1-100]`
- Recursively (or just top-level — your choice) finds image files.
- Reads each safely (check for `None`).
- Saves into `OUTPUT_DIR` with the same base name + new extension.
- Applies format-specific parameters:
- JPEG/WebP: `--quality` (default 90).
- PNG: `--compression` (default 6).
- Prints a summary:
- per-file: name, input size, output size, ratio
- total: number converted, total in/out size, average ratio

Constraints

- Use `cv2.imread / cv2.imwrite`.
- Use `argparse`.
- Skip non-image files quietly.
- Create `OUTPUT_DIR` if missing.

Expected Output (sample)

```
$ python convert.py datasets/starter/ out/ --format jpg --quality 80
```

```
astronaut.png      410.5 KB ->  68.2 KB  (16.6%)
coffee.png        487.1 KB ->  56.8 KB  (11.7%)
coins.png          58.0 KB ->  20.3 KB  (35.0%)
...
```

```
Converted 10 files
Total input : 2843.1 KB
Total output:  401.7 KB
Average ratio: 14.1%
```

Grading Rubric

Criterion	Weight
Reads + None check	15%
Correct param mapping	25%

Output dir handling	15%
Summary table	20%
Style + argparse	25%

Submission

Save as assignment_module2.py.

Module 2 — Mini Project: Image Gallery Viewer

Overview

A keyboard-driven gallery: load all images from a folder, show them one at a time, navigate with arrow keys. Shows off cv2.imshow event loops and the difference between OpenCV display and matplotlib.

Concepts Used

- cv2.imread (M2 L1)
- cv2.imshow + waitKey event loop (M2 L3)
- BGR / RGB handling (M2 L4)
- pathlib (M1 / general Python)

Features

- Reads all images in a given folder (jpg/png/bmp/webp).
- Displays one at a time with file info overlaid (cv2.putText).
- Keys:
- n / → → next image
- p / ← → previous image
- s → save the current image to ./gallery_out/
- q / Esc → quit

Starter Code

gallery.py:

```
import sys
import cv2
from pathlib import Path

EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def find_images(folder):
    return sorted(p for p in Path(folder).iterdir() if p.suffix.lower() in EXTS)

def overlay_info(img, path, idx, total):
    h, w = img.shape[:2]
    info = f"[{idx + 1}/{total}] {path.name} ({w}x{h})"
    out = img.copy()
    cv2.rectangle(out, (0, 0), (w, 30), (0, 0, 0), -1)
    cv2.putText(out, info, (10, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                (255, 255, 255), 1, cv2.LINE_AA)
    return out

def main(folder):
    paths = find_images(folder)
    if not paths:
        print("no images found in", folder); return
```



```

Path("gallery_out").mkdir(exist_ok=True)

idx = 0
while True:
    img = cv2.imread(str(paths[idx]))
    if img is None:
        print("skip (unreadable):", paths[idx]); idx = (idx + 1) % len(paths);
    continue

    cv2.imshow("gallery", overlay_info(img, paths[idx], idx, len(paths)))
    key = cv2.waitKey(0) & 0xFF

    if key in (ord('q'), 27):                # q or Esc
        break
    elif key in (ord('n'), 83):              # n or right arrow (83 on many
systems)
        idx = (idx + 1) % len(paths)
    elif key in (ord('p'), 81):              # p or left arrow (81)
        idx = (idx - 1) % len(paths)
    elif key == ord('s'):
        out = Path("gallery_out") / paths[idx].name
        cv2.imwrite(str(out), img)
        print("saved ->", out)

cv2.destroyAllWindows()

if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else "datasets/starter"
    main(folder)

```

Expected Interaction

```

$ python gallery.py datasets/starter
(window opens showing first image with overlay "[1/10] astronaut.png (512x512)")
press n -> [2/10] brick.png (512x512)
press s -> saved -> gallery_out/brick.png
press q -> window closes

```

Extension Challenges

- Add a thumbnail strip at the bottom of the window.
- Add r to rotate 90° clockwise; r again rotates more.
- Save thumbnails into gallery_out/thumbs/.

Grading Rubric

Criterion	Weight
Loads & cycles images	30%
Save key works	15%
Info overlay correct	15%
Handles unreadable files	15%
Style + structure	25%